

CEOS

INFRASTRUCTURE

A Governed Transactional Assurance Substrate for Autonomous Software Engineering

Formal Systems Architecture, Security Model, and Falsifiable Research Program

Technical Research Preprint	September 26, 2026
Project	Caivra Engineering Operating System (CEOS) Infrastructure
Organization	Caivra Tech LLC
Research domains	Distributed Systems • AI-Agent Security • Software Assurance • Formal Methods • Transactional Engineering

RESEARCH STATUS

Implemented capability, canonical planned architecture, and research hypotheses are distinguished throughout. Build closure is not treated as runtime or deployment authority.

Abstract

The increasing capability of autonomous software-engineering agents creates a systems problem that is not adequately described as AI safety alone. Once a model can invoke tools, mutate repositories, operate CI/CD systems, manipulate infrastructure, coordinate agents, and cause external effects, the central question becomes: under what exact state, authority, evidence, freshness, execution, and recovery conditions may a machine-generated action become real? This paper analyzes the Caivra Engineering Operating System (CEOS) Infrastructure as an experimental architecture for addressing that problem. CEOS models autonomous engineering as governed state-and-effect transactions rather than unrestricted sequences of model-generated tool calls. Its principal separation is between probabilistic cognition and deterministic execution authority: reasoning may propose, verification may substantiate, deterministic governance may authorize, controlled execution may mutate, and observed outcomes may become evidence. The architecture comprises a 16-layer Trust Architecture plus cross-layer protocols for assurance, context continuity, parallel workstreams, causal interaction integrity, governed adaptation, evidence, and recovery. We formalize authority as an intersection of independently limiting permissions; model capability attenuation, exact-state freshness, dispatch-time revalidation, candidate-versus-canonical separation, ambiguous external outcomes, causal evidence graphs, provenance-aware context compilation, and risk-constrained model routing. We relate these mechanisms to zero-trust architecture, least privilege, complete mediation, distributed leases, causal ordering, sagas, provenance systems, secure information-flow lattices, policy engines, and contemporary prompt-injection research. The live CEOS Infrastructure repository was inspected for this paper. At the observed canonical main revision 97b01516f78a0756cfeda396d0b4ebf488feb101, infrastructure phases I0 through I5 are recorded as closed, canonical, and post-merge validated build phases, while runtime activation, deployment, release, State032, and I6 remain separately unauthorized or inactive. The paper therefore treats CEOS as a falsifiable systems hypothesis rather than a completed proof: the decisive question is whether integrated enforcement of exact mutation, exact state, authority, bounded lease, dispatch, evidence, validation, and canonicalization materially outperforms a correctly configured incumbent composite stack.

Keywords — autonomous software engineering; AI agents; distributed systems; deterministic authorization; capability security; provenance; evidence; zero trust; formal invariants; transactional engineering; digital twins; recovery; multi-agent systems.

Research Status and Epistemic Classification

This paper distinguishes canonical CEOS facts, live repository observations, external technical facts, and analytical formalizations. Mathematical expressions are formal models of CEOS semantics unless explicitly identified as implemented machine-checked properties.

Evidence class	Meaning
Canonical CEOS fact	Defined by current canonical architecture, authority, lexicon, or policy.
Live repository observation	Directly observed in the canonical repository, source tree, pull-request history, or tests.
External technical fact	Derived from published standards or research outside CEOS.
Analytical formalization / hypothesis	A mathematical representation or proposed

interpretation that requires separate validation.

1. Introduction

Modern software-engineering agents increasingly combine probabilistic inference with the ability to invoke tools and alter real systems. A language model can generate code, inspect a repository, call a provider API, start a build, coordinate another agent, or request infrastructure mutation. The cognitive mechanism is probabilistic, while the resulting state transition is concrete. This creates a structural mismatch between uncertain reasoning and consequential execution.

CEOS begins from the proposition that the probability distribution that generated an action and the deterministic mechanism that permits the action must be separate semantic objects. A model may be uncertain; an authorization decision cannot safely mean “probably authorized.” A model may hallucinate repository state; a mutation path cannot safely treat that hallucination as current state. A model may recover from a linguistic mistake by producing another answer; a deleted production database, leaked secret, duplicated external effect, or invalid canonical promotion may not be reversible.

Accordingly, CEOS treats autonomous engineering as a governed transition system. Probabilistic intelligence may investigate, diagnose, predict, classify, critique, and propose. Deterministic CEOS mechanisms own authorization, bounded capability, freshness, state admission, protected dispatch, validation disposition, and canonicalization.

2. Research Questions

The CEOS research program can be expressed through five questions. First, can consequential machine engineering be structured so that probabilistic systems determine what to propose without becoming the mechanism that determines what is permitted? Second, can authorization be bound to the exact state against which it was established so that material drift causes revalidation? Third, can an engineering operation be represented as a governed transaction connecting intent, approval, authority, capability, dispatch, external effect, observation, validation, evidence, reconciliation, and canonicalization? Fourth, can the system generate structured evidence sufficient to reconstruct why an operation was permitted, what it acted upon, what actually occurred, and why the result did or did not become canonical? Fifth, does integrated enforcement of these properties produce materially better safety, assurance, recoverability, or operator economics than a correctly configured combination of incumbent IAM, CI/CD, source-control, policy, sandbox, observability, provenance, and workflow systems?

The fifth question remains intentionally unresolved. CEOS's own commercial falsification work rejected the broad thesis that generic AI-agent governance alone represents an unoccupied standalone category. The surviving technical thesis is narrower: integrated enforcement of exact mutation, exact state, current authority, bounded capability, dispatch freshness, evidence, validation, and canonical disposition may produce a stronger assurance boundary than fragmented composition.

3. Observed Canonical Infrastructure State

A fresh inspection of the active CEOS Infrastructure repository preceded this paper. The observed canonical repository is [cserranno85-jpg/CEOS-Infrastructure](#), branch main, at revision

97b01516f78a0756cfeda396d0b4ebf488feb101 with tree 21b688e617078bc8527e04d4987c9a56c9abbe33. The observed HEAD is the merge that performs final I5 canonical closure reconciliation.

The current build record distinguishes implementation capability from runtime authority. Infrastructure phases I0 through I5 are recorded as closed, merged, canonical-main, and post-merge validated build phases. I6, Distributed Enterprise/Federation, is dependency-valid but inactive and requires explicit owner authorization. State031 remains the selected authority state. State032, runtime activation, deployment, release, and Integration Epoch advancement remain separately unauthorized.

This distinction is fundamental. Merged code demonstrates that a capability exists in source. Green validation demonstrates that specified checks passed. Neither proposition establishes that the system is deployed, operating, healthy, or authorized to perform production mutation.

4. Canonical Sixteen-Layer Trust Architecture

CEOS preserves sixteen logical architecture layers: (1) Microkernel; (2) Intent to Work Contract; (3) Deterministic Authority Kernel; (4) Capability Leases; (5) Repository Digital Twin; (6) Cognitive Context Plane; (7) Authoritative Documentation Grounding; (8) Risk and Blast-Radius Intelligence; (9) Continuous Compliance Runtime; (10) Deterministic Context and Validation Dispatcher; (11) Governed Agentic Execution Graph; (12) Governed Agent Mesh; (13) Transactional Engineering Runtime; (14) Validation and Synthetic Validation Factory; (15) Evidence Ledger and Deterministic Replay; and (16) Token Economics and Compute Governance.

These layers are semantic ownership boundaries, not a mandate for sixteen services or sixteen synchronous network calls. The distinction matters because assurance improves when important concepts have one canonical semantic owner. Duplicating authority, state, evidence, or validation ownership can create contradictory truth rather than defense in depth.

5. Intent, Work Contracts, and Bounded Semantics

Natural-language intent is too ambiguous to serve directly as execution authority. CEOS therefore converts intended work into a bounded Work Contract. A Work Contract captures the allowed action classes, target resources, eligible environments, constraints, expiration or completion conditions, and expected result.

A useful formal representation is $W = \langle A, R, E, C, t_exp, O \rangle$, where A is the allowed action set, R the resource set, E the environment set, C constraints, t_exp expiration, and O expected outcome constraints. The Work Contract is not itself the entire authority decision; it is one independently limiting input into that decision.

This prevents a common agent-system error: allowing an ambiguous instruction such as “fix it” to expand opportunistically into any action the current tool credential happens to permit.

6. Deterministic Authority Kernel

The Authority Kernel is the semantic owner of protected execution-admission decisions. Its central property is attenuation: effective permission is formed by intersection, not by accumulation of loosely related grants. A general CEOS-compatible expression is:

$$P_{\text{effective}}(q, S_t) = W \cap R \cap D \cap C_s \cap P \cap F \cap T \cap E \cap H,$$

where W is Work Contract authorization, R role permission, D dependency access, C_s current-state eligibility, P policy eligibility, F freshness, T target eligibility, E environment eligibility, and H any required human approval.

In Boolean form, $\text{Auth}(q, S_t) = \text{AND}_i g_i(q, S_t)$. Any required false gate denies the protected action. This produces a monotonic restriction property: adding a restrictive gate cannot expand authority. A credential may technically enable a broad operation while CEOS authority remains narrower. Credential possession therefore does not imply authority.

7. Capability Leases and Dispatch-Time Freshness

A Capability Lease represents temporarily usable, bounded permission. It may be modeled as $\Lambda = \langle p, A, R, E, \sigma, g, t_0, t_{\text{exp}}, \Gamma \rangle$, containing principal, allowed operations, resources, environment, state fingerprint, generation or fencing value, issue time, expiry, and additional constraints.

Dispatch is admissible only if current authority remains true, the request remains within lease scope, the lease has not expired or been revoked, and the lease remains fresh relative to the current material state.

This design addresses the time-of-check/time-of-use problem. Let S_{ta} be the state used during authorization and S_{td} the state at dispatch. For the projection π_m containing dimensions material to the action, CEOS aims to preserve the invariant: if $\pi_m(S_{ta})$ differs materially from $\pi_m(S_{td})$, dispatch must be withheld or authorization recomputed against the new state.

8. Stable Current-Authority Resolution

CEOS applies freshness and anti-rollback logic to its own authority corpus. The stable current-authority resolver selects an immutable authority state through exact identity, sequence, digest, and size checks. State031 is currently selected with sequence 31 and a fixed SHA-256 digest.

The important property is not the particular digest. It is the resolution rule: current authority is not inferred from whichever file looks newest, from a model's memory, or from a mutable user interface. If head verification fails, current authority becomes unresolved and protected operations fail closed.

This is conceptually related to secure metadata systems such as The Update Framework, which bind version, hashes, and freshness to prevent rollback and stale-state acceptance. CEOS applies analogous reasoning to engineering authority rather than package metadata.

9. Repository Digital Twin

The Repository Digital Twin is a structured representation of the engineering state against which decisions are made. A useful state vector is $S_t = \langle R_t, T_t, D_t, E_t, P_t, I_t, V_t, C_t \rangle$, representing repository revision, tree fingerprint, dependency state, environment state, policy revision, Integration Epoch, validation state, and causal/dependency state.

The Twin is not automatically authoritative merely because it is detailed. CEOS distinguishes claimed, observed, validated, and canonical status. A provider's mergeable flag, a model's statement, or a UI projection is an observation or claim, not necessarily canonical truth.

This distinction permits CEOS to reason counterfactually without overwriting reality. Predicted state may inform risk and planning, but simulation does not become observation.

10. Cognitive Context Plane and Minimal Sufficient Context

Large language models require context, but indiscriminate context expansion increases cost, latency, contamination risk, and epistemic ambiguity. CEOS therefore favors minimal sufficient context with provenance preservation.

The context problem can be modeled as an optimization: choose C^* from the available context universe to minimize token cost, latency cost, and contamination risk, subject to task sufficiency, inclusion of applicable authority and negative constraints, and preservation of provenance.

The future Context Lifecycle and Continuity Governor extends this into governed checkpointing, rollover, context-pressure accounting, and provider-neutral continuity. The architectural principle is that conversation history is disposable; durable CEOS state is not.

11. Authoritative Documentation Grounding

CEOS treats information sources as typed objects with differing authority rather than one flat retrieval corpus. A current authority file, canonical architecture contract, historical handoff, research report, external article, generated summary, and model inference may all be relevant while carrying different semantic force.

The resulting architecture resembles an epistemic type system: retrievable does not mean authoritative, and a summary of an authoritative document does not automatically inherit the document's authority. This distinction becomes critical when autonomous agents consume untrusted or stale content.

12. Risk and Blast-Radius Intelligence

Syntactic change size is not a reliable measure of engineering consequence. One line can alter a security boundary, while a large documentation patch can have no executable effect. CEOS therefore reasons about semantic blast radius.

Let $G=(V,E)$ be a dependency graph. For mutation m , $D_0(m)$ is the directly touched set and $D_{\{k+1\}}(m)$ expands through dependency edges. A weighted blast measure can be modeled as $BR(m)=\sum_{v \in \text{Reach}(m)} w_v c_v p_v$, combining criticality, consequence, and applicability or propagation. This formula is an analytical formalization rather than a current canonical scoring equation.

The important system property is that risk and assurance depth depend on consequence and dependency structure, not only on line count or model confidence.

13. Continuous Compliance and Policy

Compliance in CEOS participates in admission and validation rather than existing only as an after-the-fact report. Applicable requirements can therefore constrain execution before side effects occur.

This is compatible with policy-as-code systems such as Open Policy Agent and Cedar, which separate policy decisions from application enforcement. CEOS treats policy as necessary but not sufficient: a policy permit does not independently establish current authority, lease freshness, exact state, validation sufficiency, or canonicity.

14. Dispatcher and Adaptive Assurance

The deterministic dispatcher coordinates eligible work without becoming a second authority engine. Its inputs include current state, authority, valid capability, validation requirements, and risk.

CEOS's Adaptive Assurance and Latency Policy separates model difficulty from operational risk. A difficult read-only theorem problem may need a strong model but low operational assurance. A trivial shell command may be extremely high risk because it has irreversible side effects.

Accordingly, AssuranceDepth = $f(\text{Impact}, \text{Irreversibility}, \text{Authority}, \text{BlastRadius}, \text{StateUncertainty}, \text{EvidenceRequirement})$, not $f(\text{ModelIntelligence})$. Model sophistication cannot substitute for authority or evidence.

15. Governed Agentic Execution Graph and Agent Mesh

The Governed Agentic Execution Graph represents work topology; the Governed Agent Mesh represents eligible participants capable of performing work. They are deliberately distinct.

An execution graph can be represented as a directed acyclic graph $G_x=(N,E)$, with dependency relation $(u,v) \in E$ implying u precedes v . This maps naturally to causal partial-order reasoning in distributed systems.

Agent assignment, however, does not imply authority. A scheduler may choose a worker and a router may choose a model, but neither selection creates permission. Delegation must narrow or preserve authority, never widen it.

16. Transactional Engineering Runtime

CEOS replaces the conventional Think → Act → Observe agent loop with a richer governed lifecycle:

Intent → Admissibility → Current State → Authorization → Freshness → Bounded Capability → Dispatch → External Effect → Observation → Validation → Evidence → Reconciliation → Canonical Disposition.

The objective is not to claim database-style atomicity across arbitrary providers. Instead, the architecture makes uncertainty explicit and keeps attempted execution, provider acknowledgement, observed effect, validation, and canonical promotion as separate states.

This resembles long-running transaction theory and sagas, where partial external work may require compensating actions. CEOS adds the rule that compensation is a new causal action; it does not erase historical reality.

17. UNKNOWN_OUTCOME and Recovery

Distributed systems often lose the response to an operation whose external effect may or may not have occurred. CEOS treats this as a first-class UNKNOWN_OUTCOME state rather than assuming failure.

If $D=1$ denotes dispatch, $K_s=0$ means success is unproven, and $K_f=0$ means failure is unproven, then $UNKNOWN_OUTCOME = D \wedge \neg K_s \wedge \neg K_f$.

The unsafe strategy is timeout $\rightarrow$ retry. The safer CEOS strategy is timeout $\rightarrow$ observe $\rightarrow$ reconcile $\rightarrow$ classify $\rightarrow$ complete, compensate, retry only if proven safe, or remain unresolved. This is essential for non-idempotent effects and for crash recovery after dispatch.

18. Candidate State, Validation, and Canonicalization

CEOS deliberately prevents state names from implying one another. SEALED does not imply VALIDATED. VALIDATED does not imply CANONICAL. APPROVED does not automatically imply AUTHORIZED. DISPATCHED does not imply EXECUTED. EXECUTED does not imply CANONICAL.

Canonicalization is an explicit admission event. A candidate becomes canonical only when the applicable canonical admission mechanism succeeds against current state and required evidence. A successful API response is therefore not equivalent to canonical truth.

This semantic discipline removes a class of distributed-control errors caused by human-language intuition leaking into machine state transitions.

19. Parallel Workstreams and Canonical Promotion

CEOS permits parallel isolated work while serializing shared-state transitions and canonical promotion. File non-overlap is insufficient to prove semantic independence because two changes can modify different files while affecting the same schema, authority rule, API, migration assumption, or architectural decision.

A candidate validated against an earlier base can become stale for integration after canonical state advances. Staleness does not automatically mean the candidate is defective; it means reconciliation is required.

The intended invariant is that competing promotions cannot both commit under the same fenced canonical generation. This gives canonical promotion a clear serialization boundary while preserving concurrency elsewhere.

20. Validation and Assurance Coverage

A green validator proves only that the checks it actually executed passed. It does not prove that every important invariant was tested. CEOS therefore distinguishes validation execution from assurance coverage.

Let $\Omega=\{\omega_1,...,\omega_n\}$ be the set of proof obligations. For each obligation define $v_i=1$ if current applicable evidence exists and 0 otherwise. A weighted analytical coverage measure is $Coverage=(\sum w_i v_i)/(\sum w_i)$. CEOS should not necessarily collapse assurance into one scalar because a critical missing proof must not be averaged away by many trivial passes.

The system-level research objective is therefore to connect validation evidence to explicit obligations and to prevent stale or non-applicable evidence from silently satisfying current requirements.

21. Evidence Ledger and Deterministic Replay

CEOS treats evidence as an architectural object rather than logging exhaust. A causal evidence graph may contain actor, request, Work Contract, approval, authority, lease, state, dispatch, provider invocation, effect, observation, validation, reconciliation, and canonicalization nodes connected by derived-from, authorized-by, executed-as, observed-by, and validated-by relations.

A successful forensic reconstruction should permit traversal from actor and intent through authority and effect to validation and disposition. This is broader than ordinary software supply-chain provenance: it attempts to explain not only how an artifact was produced, but why the state transition was permitted and how its result became accepted or withheld.

22. Token Economics and Compute Governance

Autonomous engineering can fail economically even when logically correct. Multi-agent workflows can recursively spawn expensive reasoning, retrieval, verification, testing, and human review.

A useful total-cost model is

$C_{total} = C_{model} + C_{compute} + C_{storage} + C_{network} + C_{human} + C_{validation} + C_{recovery}$. The engineering objective is not simply to minimize tokens; it is to maximize verified successful outcomes per unit of total resource cost while preserving safety and assurance constraints.

This makes performance a governed resource. A latency optimization that weakens authority, freshness, or validation is a regression even if it makes the interface faster.

23. Cross-System Interaction Integrity

Engineering effects propagate across systems. A repository promotion may invalidate another workstream's base, a dependency assumption, an approval, a validation result, or a deployment plan. CEOS's cross-system interaction model therefore uses causal identity, declared dependencies, prechecks, monitored execution, and postcondition reconciliation.

A Causal Action Envelope can be represented as

$CAE = \langle id, cause, principal, W, Auth, \Delta, S_{pre}, A, S_{expected}, Risk, Evidence \rangle$. The lifecycle becomes $Precheck(CAE) \rightarrow Execute(CAE) \rightarrow Observe \rightarrow PostconditionReconcile$.

The critical principle is that expected downstream state is not assumed merely because the caller intended it.

24. Governed Reasoning Plane

CEOS has adopted a future provider-neutral Governed Reasoning and Intelligence capability without creating a seventeenth layer or transferring authority to models. The intended flow is Request $\rightarrow$ Context Compiler $\rightarrow$ Reasoning Router $\rightarrow$ Model $\rightarrow$ Reasoning Artifact $\rightarrow$ Verification $\rightarrow$ Authority Reevaluation $\rightarrow$ Controlled Execution.

Models may investigate, classify, hypothesize, plan, review, critique, predict, and propose. They may not mint capability leases, grant themselves authority, expand scope, override revocation, declare stale authority current, directly overwrite canonical state, or convert confidence into authorization.

For a fixed requested action and identical authoritative inputs, substituting one model for another must not independently change the deterministic permission result. The model may change the proposal; it may not change the laws governing whether that proposal is permitted.

25. Prompt Injection and Compromised-Model Assumption

Tool-using agents face a structural prompt-injection problem when untrusted external data is placed in the same reasoning context as privileged instructions. Modern research such as AgentDojo and CaMeL supports an architectural conclusion: systems should not assume the model will always distinguish data from malicious instructions.

CEOS therefore benefits from treating the reasoner as potentially compromised. Retrieved content should not become instruction-eligible merely because it appears in context. Authority must be determined by CEOS-owned semantics outside the model's generative reasoning.

This is stronger than asking the model to ignore malicious prompts; it aims to ensure that successful prompt injection still cannot manufacture permission.

26. Verification Is Not Model Consensus

A second model can provide useful independent criticism, but asking multiple correlated models to agree is not equivalent to proof. Consensus_models does not imply truth.

CEOS therefore favors heterogeneous verification: schemas, deterministic policy, exact evidence resolution, static analysis, type checking, security scanning, unit and integration tests, property tests, Digital Twin consistency, dependency analysis, counterfactual simulation, independent model review, and human adjudication where required.

Verification strength should depend on evidence diversity and applicability, not simply the number of agreeing language-model samples.

27. Governed Model Routing

The future reasoning system can route tasks to the least expensive eligible model that satisfies risk, privacy, residency, latency, and quality requirements. Formally, choose $m^* = \operatorname{argmin} \operatorname{Cost}(m, q)$ over eligible models subject to $Q(m, q) \geq Q_{\min}(\operatorname{Risk}(q))$, privacy eligibility, residency eligibility, latency constraints, and governance eligibility.

This places model optimization inside the governance envelope. The architecture rejects both “always use the strongest model” and “always use the cheapest model.”

28. Authority Attenuation Calculus

A future formal Authority Attenuation Calculus can model delegation. If parent capability $C_p = (A_p, R_p, E_p, t_p)$, then child capability C_c must satisfy $A_c \leq A_p$, $R_c \leq R_p$, $E_c \leq E_p$, and $t_c \leq t_p$. Write $C_c \leq C_p$.

The governing invariant is $\operatorname{Delegate}(C_p) \rightarrow C_c$ implies $C_c \leq C_p$. For a delegation chain $C_0 \rightarrow C_1 \rightarrow \dots \rightarrow C_n$, authority must monotonically attenuate: $C_n \leq \dots \leq C_1 \leq C_0$.

This becomes particularly important for large agent swarms, where recursive delegation can otherwise create accidental privilege amplification.

29. Provenance-Carrying Information Lattice

A future Provenance-Carrying Information Lattice can attach source, trust, freshness, sensitivity, instruction eligibility, and authority class to material information. For an unverified transformation $x_1, \dots, x_n \rightarrow y$, trust should not spontaneously increase: $\text{Trust}(y) \leq \min_i \text{Trust}(x_i)$, unless an explicit verification or declassification mechanism justifies promotion.

This extends classical lattice-based information-flow reasoning into engineering epistemics. A model-generated summary of an authoritative source can remain useful while retaining a derived status rather than silently inheriting the source's authority.

30. Attested Enforcement Assurance

A policy can claim that an operation is sandboxed, isolated, read-only, or non-networked. That claim is not equivalent to runtime enforcement.

Let G_c be claimed guarantees and G_e guarantees actually established by the execution substrate. A safety claim is admissible only where $G_c \subseteq G_e$. If the requested guarantee exceeds the substrate's physical capability, the system should fail closed or explicitly downgrade the claim rather than pretending the enforcement exists.

This future contract is intended to bind policy language to attested runtime reality.

31. Effect Commit Protocol

For consequential external effects, CEOS ultimately needs precise boundaries among attempt, acknowledgement, effect, observation, validation, and canonical disposition:

Attempt $\neq$ Ack $\neq$ Effect $\neq$ Observed $\neq$ Validated $\neq$ Canonical.

This distinction is necessary because heterogeneous external systems rarely provide a universal exactly-once transaction boundary. The Effect Commit Protocol is therefore a planned mechanism for making the state of external side effects explicit and recoverable.

32. Counterfactual Policy Certification

The Digital Twin permits reasoning about predicted successor state $\hat{S}_{t+1} = T_{\text{hat}}(S_t, a)$. A counterfactual policy engine can ask whether the predicted state would violate policy before executing.

However, predicted state is not observed state. Simulation contributes evidence; it does not become authority or overwrite reality. Counterfactual certification is therefore an advisory or gating input whose predictions must later be reconciled against actual effects.

33. System Health as a Vector

CEOS's emerging health concept should be multidimensional rather than a single green indicator. A health vector $H = [h_A, h_S, h_V, h_E, h_D, h_R, h_C, h_P]^T$ can represent authority integrity, state freshness,

validation assurance, evidence integrity, dependency health, recovery readiness, containment/security, and performance/resource condition.

A system can therefore have healthy latency while authority integrity is invalid. Overall health must not average away a critical failed dimension. Telemetry that looks healthy is not itself proof of correctness.

34. Lead-Lag Engineering Pipeline

CEOS development employs a two-track methodology in which the canonical build proceeds at phase N while a shadow advanced-engineering track investigates approximately phase N+1. The shadow track can prototype, falsify, repair, adversarially test, and accumulate evidence, but shadow success is not canonical acceptance.

The intended economic hypothesis is that future canonical work can shift from discover-design-debug toward adjudicate-implement-validate. Whether the method actually reduces cycle time or defect rate is measurable and should itself be tested rather than assumed.

35. Infrastructure Product Topology

The target topology is Human/Automation → API/CLI/SDK/Operator Surface → Work Contract + Identity + Policy + Authority + Lease + State Admission → Dispatcher + Execution Graph + Agent Mesh ↔ Models/Agents → Transactional Execution → Workers/Tools/Repositories/CI/Cloud/OS → Observation + Validation + Evidence + Reconciliation.

The live repository contains corresponding infrastructure-oriented components including core services, a local API, CLI surfaces, provider-neutral boundaries, host/worker contracts, dispatch tests, reconciliation tests, operator projections, freshness checks, and an MCP/ChatGPT integration surface. These are repository capabilities, not proof of authorized live production operation.

36. I6 and the Distributed Failure Model

I6, Distributed Enterprise/Federation, is the next dependency-valid major phase but is not currently authorized for implementation. Its target concerns include remote workers, multi-host control-plane semantics, PostgreSQL/S3 adapters where justified, OIDC/workload identity, mTLS, tenant isolation, distributed fencing, causal integrity, recovery, and enterprise policy integration.

This phase changes the dominant failure model. Local concurrency becomes distributed concurrency. Network partitions become material. Process identity becomes workload identity. Lease correctness must survive disconnection, clock skew, worker death, and restart. Evidence must traverse machine boundaries without losing causal binding. Canonical promotion must resist split-brain behavior.

37. Security Model

A CEOS threat model should assume that any subset of models, agents, prompts, retrieved context, providers, workers, credentials, caches, stale authority records, evidence objects, repository state, or networks can be faulty or adversarial.

The architectural objective is therefore not “trust the internal component.” It is “trust only the invariant established by the owning mechanism.” An authenticated worker is not automatically authorized. A powerful model is not automatically trustworthy. A green CI result is not merge authority. A provider acknowledgement is not canonical state.

38. Candidate Formal Safety Invariants

Several properties are candidates for future model checking or theorem-oriented specification:

- A. $\text{Dispatch}(q,t) \Rightarrow \text{Auth}(q,S_t)$.
- B. $\text{MaterialDrift}(S_a,S_d) \Rightarrow \neg \text{Dispatch} \vee \text{Reauthorize}$.
- C. $\text{Delegate}(C_p)=C_c \Rightarrow C_c \leq C_p$.
- D. For fixed request and authoritative state, model substitution cannot mint authority.
- E. $\text{SEALED}(x)$ does not imply $\text{CANONICAL}(x)$.
- F. $\text{UNKNOWN_OUTCOME}(x) \wedge \neg \text{SafeRetryProof}(x) \Rightarrow \neg \text{Retry}(x)$.
- G. Competing canonical promotions cannot both commit under the same fencing generation.
- H. $\text{Evidence}(x)$ does not imply $\text{Authorization}(x)$.
- I. $\text{ValidationPass}(x)$ does not imply $\text{Canonical}(x)$.
- J. Simulated state cannot overwrite observed authoritative state absent an explicit governed transition.

These invariants convert CEOS doctrine into falsifiable machine properties.

39. Relationship to Existing Systems Science

CEOS should not claim primitive novelty where strong precedent exists. Least privilege and complete mediation inform its authority semantics. NIST Zero Trust informs the rejection of implicit trust. OPA and Cedar demonstrate policy decision/enforcement separation. Distributed leases inform bounded authority. Lamport's happens-before relation informs causal ordering. Sagas inform long-running compensation. Linearizability informs serialized shared-state transitions. SLSA and in-toto inform provenance. Denning's lattice model informs information-flow reasoning. Contemporary agent-security research informs the compromised-model assumption.

The plausible CEOS contribution is therefore compositional: enforcing these concepts as one exact-state engineering transaction lifecycle rather than relying on loosely coupled operator convention.

40. Integrated-Invariant Hypothesis

Let $G=\{m,s,a,f,l,d,e,v,c\}$ denote exact mutation, state, authority, freshness, lease, dispatch, evidence, validation, and canonical disposition. The CEOS hypothesis is that enforcing a relation $\Phi(m,s,a,f,l,d,e,v,c)$ as one governed lifecycle provides stronger assurance or lower assurance-integration burden than implementations where these properties are independently configured across multiple systems.

This remains a hypothesis. Architectural elegance is not evidence of superiority.

41. Null and Alternative Hypotheses

The eventual benchmark should test a fair incumbent composite.

H0: a correctly configured incumbent stack achieves semantic equivalence with CEOS at reasonable implementation, operational, performance, and assurance cost.

H1: CEOS produces a material improvement in one or more governed outcomes—such as unsafe-permit rate, stale-authority prevention, reconstruction completeness, recovery correctness, operator burden, or assurance-adjusted cost—without unacceptable regressions elsewhere.

The incumbent must not be deliberately weakened. A meaningful scientific result requires comparison against strong available alternatives.

42. Required Adversarial Experiments

The benchmark corpus should include stale-authority execution after material state drift; revocation races; credential overreach; prompt-injected reasoners; ambiguous external outcomes; concurrent canonical promotion; forged or non-applicable evidence; restart between dispatch and observation; stale leases; provider timeouts; duplicate execution; partial persistence; and alternate lower-layer entry points.

Each test should define the invariant, trigger, expected safe behavior, evidence required to establish the result, and a falsification condition. Tests designed only to pass do not establish the architecture's strength.

43. Benchmark Metrics

Useful metrics include unsafe permit rate, false denial rate, stale-authority escape rate, evidence reconstruction completeness, reconstruction time, manual intervention count, governance latency overhead, recovery success, and assurance-adjusted cost.

A system that blocks every action trivially avoids unsafe execution but is unusable. The benchmark is therefore multi-objective: minimize unsafe permits, false denials, operator burden, latency, and cost subject to required correctness and assurance.

44. Formal Verification Program

The deterministic core is sufficiently state-oriented to justify a future TLA+-style or equivalent formal specification. Candidate variables include state, work, approval, authority, lease, revocation, dispatch, effect, observation, validation, evidence, and canonical disposition.

Safety properties can assert that dispatch always implies current authority and that canonical state always implies valid canonical admission. Liveness properties can test whether eligible work eventually resolves under explicit non-failure assumptions. Formal methods become especially valuable as I6 and I7 introduce distributed partitions, remote workers, and stronger fencing requirements.

45. NIST Alignment and Secure Development

CEOS is not automatically NIST compliant, but its lifecycle has conceptual alignment with the NIST AI Risk Management Framework: Govern maps to authority, policy, and Work Contracts; Map to the Twin,

context, dependencies, and blast radius; Measure to validation, evidence, and health; Manage to admission, containment, reconciliation, and recovery.

Likewise, NIST's Secure Software Development Framework provides a useful external reference for integrating security into development lifecycles. CEOS extends the problem into machine-executed software development, where the process that changes software must itself become governed.

46. Performance, Complexity, and Non-Bloat

Logical layers need not become serial network tollbooths. Many checks can be local, parallel, cached under explicit freshness, or compiled into a single decision. The performance objective is to minimize governance latency subject to safety and assurance constraints.

Complexity is itself a risk. Creating a second authority engine, second evidence ledger, second scheduler, or second Digital Twin can produce contradictory truth. For each important semantic concept x , the preferred architecture is $|\text{Owner}(x)|=1$, with many consumers but one canonical owner.

Non-bloat is therefore not merely aesthetic; it is an assurance property.

47. Limitations

The present work has material limitations. Source and validation evidence are not production evidence. Several future theories remain planned rather than physically enforced. End-to-end non-bypassability is difficult to prove. External systems weaken transactional guarantees. The mathematical models in this paper are candidate specifications and are not themselves implementation proofs. Distributed federation is not yet active. Commercial value remains unproven. Agent-security research continues to demonstrate that tool-using language models are vulnerable to adversarial instruction patterns.

These limitations are not peripheral caveats; they define the boundary of the present scientific claim.

48. Future Research Agenda

The highest-value next research is proof rather than architectural ornamentation. Priorities include mechanically specifying the Authority/Lease/Dispatch state machine; model-checking stale-authority and revocation races; establishing executable capability attenuation; proving dispatch non-bypassability; implementing provenance-bound context transformations; conducting the incumbent-vs-CEOS governed-outcome benchmark; exercising real UNKNOWN_OUTCOME recovery; testing distributed fencing under partition and restart; adversarially compromising reasoning models while verifying authority containment; measuring lead-lag development against sequential development; and formalizing the multidimensional Health Matrix.

The canonical failure methodology already supports this program: reproduce causally, isolate the first incorrect transition, identify the semantic owner, and design tests capable of falsifying the repair.

49. Scientific Interpretation

CEOS can be interpreted as an engineering reference monitor. Traditional authorization asks whether a principal may perform an action on a resource. CEOS asks a larger question involving principal, intent,

exact mutation, exact repository state, environment, policy, approval, authority, lease, freshness, risk, dependency state, execution substrate, and validation obligations.

After execution it asks additional questions: did the effect actually occur; was the observed result valid; can the causal history be reconstructed; and should the result become canonical?

The system therefore attempts to make engineering intent, permission, action, effect, evidence, and truth separate but composable machine objects.

50. Conclusion

CEOS Infrastructure is an ambitious systems hypothesis for autonomous software engineering. Its defensible claim is not that it invented authorization, provenance, leases, workflow DAGs, policy engines, digital twins, validation, recovery, or model routing. The interesting proposition lies in their composition around an exact-state governed transaction.

The central theoretical law is authority conservation: no downstream mechanism may create more legitimate authority than was explicitly established upstream. Models cannot create it. Credentials cannot create it. Tools cannot create it. Workers cannot create it. Leases cannot expand it. Delegation cannot amplify it. Validation cannot retroactively create it. Evidence cannot create it. Successful execution cannot create it. Simulation cannot create it. Canonicalization requires an explicit governed admission mechanism.

The central transactional contribution is the replacement of Think → Act → Observe with Intent → Admissibility → State Resolution → Authority → Lease → Freshness → Dispatch → Effect → Observation → Validation → Evidence → Reconciliation → Canonical Disposition.

The decisive milestone is not another architectural diagram. It is an adversarial experiment against a correctly configured incumbent composite. If CEOS cannot materially outperform that baseline on safety, stale-authority prevention, evidence reconstruction, recovery, operator burden, performance, complexity, or assurance cost, the differentiation hypothesis must weaken. If it can, and the result survives replication, CEOS will have demonstrated a more consequential result: that highly autonomous software engineering can be placed inside a deterministic, evidence-carrying, exact-state transactional substrate without making probabilistic machine intelligence sovereign over the state it is permitted to change.

Appendix A. Core Mathematical Relations

Relation	Formalization
Authority intersection	$P_{\text{effective}}(q, S_t) = W \cap R \cap D \cap C_s \cap P \cap F \cap T \cap E \cap H$
Boolean admission	$\text{Auth}(q, S_t) = \wedge_i g_i(q, S_t)$
Freshness invariant	$\pi_m(S_{ta}) \neq \pi_m(S_{td}) \Rightarrow \neg \text{Dispatch} \vee \text{Reauthorize}(S_{td})$
Lease admissibility	$\text{Dispatch}(q, \Lambda, S_t) = \text{Auth} \wedge \text{InScope} \wedge \neg \text{Expired} \wedge \neg \text{Revoked} \wedge \text{Fresh}$
Authority attenuation	$\text{Delegate}(C_p) = C_c \Rightarrow C_c \leq C_p$
Unknown outcome	$\text{UNKNOWN_OUTCOME} = D \wedge \neg K_s \wedge \neg K_f$
Context optimization	$C^* = \text{argmin} [\text{TokenCost} + \lambda_1 \text{Latency} + \lambda_2 \text{ContaminationRisk}], \text{ subject to sufficiency and}$

	provenance
Model routing	$m^* = \operatorname{argmin} \operatorname{Cost}(m,q)$, subject to quality, privacy, residency, latency, and governance eligibility
Evidence graph	Actor $\rightarrow$ Intent $\rightarrow$ Contract $\rightarrow$ Authority $\rightarrow$ Lease $\rightarrow$ Dispatch $\rightarrow$ Effect $\rightarrow$ Observation $\rightarrow$ Validation $\rightarrow$ Disposition
Integrated invariant	$\Phi(m,s,a,f,l,d,e,v,c)$

References

- [1] CEOS Infrastructure. Canonical Post-V6 Final Architecture; Current Authority State; Master Build Plan; architectural amendments; validation and failure-reproduction policies. Active private repository, inspected September 26, 2026.
- [2] Saltzer, J. H., & Schroeder, M. D. (1975). The Protection of Information in Computer Systems. *Proceedings of the IEEE*, 63(9), 1278-1308.
- [3] Rose, S., Borchert, O., Mitchell, S., & Connelly, S. (2020). NIST SP 800-207: Zero Trust Architecture. National Institute of Standards and Technology.
- [4] Gray, C. G., & Cheriton, D. R. (1989). Leases: An Efficient Fault-Tolerant Mechanism for Distributed File Cache Consistency. *SOSP*.
- [5] Lamport, L. (1978). Time, Clocks, and the Ordering of Events in a Distributed System. *Communications of the ACM*, 21(7), 558-565.
- [6] Herlihy, M. P., & Wing, J. M. (1990). Linearizability: A Correctness Condition for Concurrent Objects. *ACM TOPLAS*, 12(3), 463-492.
- [7] Garcia-Molina, H., & Salem, K. (1987). Sagas. *ACM SIGMOD*.
- [8] Denning, D. E. (1976). A Lattice Model of Secure Information Flow. *Communications of the ACM*, 19(5), 236-243.
- [9] The Update Framework (TUF). Metadata, freshness, rollback, and freeze-attack protections.
- [10] SLSA v1.2. Provenance specification for software artifacts.
- [11] in-toto. A framework for securing the integrity of software supply chains.
- [12] Open Policy Agent. Policy decision and enforcement architecture.
- [13] Cedar Policy Language. Authorization semantics for principal, action, resource, and context.
- [14] Debenedetti, E., et al. (2024). AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. *arXiv:2406.13352*.
- [15] Debenedetti, E., et al. (2025). Defeating Prompt Injections by Design / CaMeL. *arXiv:2503.18813*.
- [16] NIST. AI Risk Management Framework and Playbook.
- [17] NIST SP 800-218. Secure Software Development Framework (SSDF) Version 1.1.